

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский ядерный университет «МИФИ»



**Институт
интеллектуальных кибернетических систем
Кафедра №22 «Кибернетика»**

Система автоматизированного развёртывания окружений

Взаимодействие со сторонними программами

Выполнили:

Кузнецова А. А.

Куренный Г. И.

Левченко А. А.

Сысоев К. В.

Чен П. Я.

Б20-504

Деревянко А.К.

Б20-564

Проверяющий:

Козырев В. П.

Содержание

Взаимодействие со сторонними программами	3
1. Взаимодействие с GitLab	3
1.1. Проверка доступности репозитория	3
1.2. Получение сообщение об обновлении репозитория	3
2. Взаимодействие с Kubernetes	3
2.1 Проверка доступности кластера	4
2.2. Сообщение кластеру о необходимости развернуть проект	4
2.3. Получение данных о развёртывании и артефактов сборки	6
2.4. Получение метрик по развёрнутому проекту	6
2.5. Изменение физических ресурсов, выделяемых на проекта	7

Взаимодействие со сторонними программами

В данном разделе описаны интерфейсы, команды и алгоритмы взаимодействия с внешними по отношению к разрабатываемому ПО системами и программами. А именно взаимодействие с GitLab и Взаимодействие с Kubernetes.

1. Взаимодействие с GitLab

1.1. Проверка доступности репозитория

Для проверки доступности репозитория необходимо выполнить команду:

```
git clone <repository-address>
```

Здесь *<repository-address>* это адрес репозитория

1.2. Получение сообщения об обновлении репозитория

Для получения сообщения об обновлении репозитория необходимо создать в данном репозитории файл с названием *.gitlab-ci.yml* и следующим содержанием:

```
deploy-job:
```

```
stage: deploy
```

```
script:
```

```
- echo "Send message"
```

Здесь в последней строке вместо *echo "Send message"* необходимо вставить *bash*-скрипт, реализующий отправку сообщения об обновлении репозитория, например с помощью команды *curl*

2. Взаимодействие с Kubernetes

Для подключения к кластеру Kubernetes используется описание конфигурации с помощью файлов *kubeconfig*, это текстовый файл, формат которого описан в [документации Kubernetes](#). Пример такого файла:

```
apiVersion: v1
```

```
clusters:
```

```
- cluster:
```

```
certificate-authority-data: <ca-data-here>
```

```
server: https://your-k8s-cluster.com
```

```
name: <cluster-name>
```

```
contexts:
```

```
- context:
    cluster: <cluster-name>
    user: <cluster-name-user>

    name: <cluster-name>
current-context: <cluster-name>
kind: Config
preferences: {}
users:
- name: <cluster-name-user>
    user:
        token: <secret-token-here>
```

В данном файле:

1. **certificate-authority-data:** УЦ кластера
2. **server:** Адрес кластера
3. **name:** Имя кластера
4. **user:** имя пользователя
5. **token:** секретный токен для выполнения действий в кластере

Взаимодействие с кластером осуществляется с помощью консольной утилиты `kubectl`, порядок установки этой утилиты описан в [документации kubectl](#).

В Kubernetes термин "под" (pod) означает самую маленькую и базовую единицу развертывания, которая может быть создана и управляема в кластере.

2.1 Проверка доступности кластера

Проверка доступности кластера осуществляется с помощью команды `kubectl cluster-info`. В случае, если кластер доступен, то вывод команды будет иметь следующий вид:

```
Kubernetes control plane is running at https://your-k8s-cluster.com
CoreDNS is running at https://your-k8s-cluster.com/api/kube-dns:dns/proxy
Metrics-server is running at https://your-k8s-cluster.com/api/https:metrics-server:https/proxy
...
```

В случае, если команда завершится с ошибкой, это означает, что кластер недоступен

2.2. Сообщение кластеру о необходимости развернуть проект

Для развертывания проекта в кластере Kubernetes необходимо выполнить следующее:

1. Установить утилиту werf. Инструкцию по установке werf можно найти в официальной документации.
2. Создать файл с названием werf.yaml в корневом каталоге проекта. В этом файле нужно определить настройки развертывания (правила написания данного файла см. в конце данного пункта)
3. Использовать команду werf build. Эта команда будет использовать настройки из файла werf.yaml

Ниже приведен пример простейшего файла werf.yaml:

project: my-app

image: my-app:latest

stages:

- from: artifacts

beforeSetup:

- go mod download

artifacts:

- from: ./app

to: /app

- from: git

beforeSetup:

- go mod download

git:

repo: https://github.com/myrepo/myapp.git

branch: master

context: ./myapp

dockerfile: ./myapp/Dockerfile

В этом примере:

- *project* указывает на имя проекта, в данном случае "my-app".
- *image* задает имя образа, в данном случае "my-app:latest".
- *stages* определяет этапы сборки образа.
- *stages.from* указывает исходные артефакты для этапа сборки.
- *stages.from: artifacts* указывает использование локальных артефактов как исходных.
- *stages.from: git* указывает использование репозитория Git как исходного.

- `stages.beforeSetup` указывает команды, которые будут выполнены перед сборкой проекта.
- `stages.artifacts` определяет артефакты сборки, которые будут добавлены в образ.
- `stages.artifacts.from` указывает путь к локальным артефактам.
- `stages.artifacts.to` указывает путь внутри образа, куда будут скопированы артефакты.
- `stages.git` указывает параметры репозитория Git.
- `stages.context` указывает каталог с исходными файлами проекта.
- `stages.dockerfile` указывает путь к файлу Dockerfile.

Подробную информацию о формате файла `werf.yaml` и доступных параметрах можно найти в [официальной документации werf](#).

2.3. Получение данных о развёртывании и артефактов сборки

Для получения артефактов после развёртывания проекта в Kubernetes с помощью консоли необходимо использовать команду `kubectl cp`.

Синтаксис команды `kubectl cp` выглядит следующим образом:

`kubectl cp <имя-пода>:<путь-в-ноде> <локальный-путь>`

где:

- `<имя-пода>` - это имя пода, в котором развернуты приложения или сервисы;
- `<путь-в-ноде>` - это путь к артефакту, который необходимо получить;
- `<локальный-путь>` - это путь, куда будет скопирован артефакт из пода.

Пример:

`kubectl cp my-pod:/var/log/app.log ./app.log`

В этом примере файл `app.log` копируется из пода с именем `my-pod` и пути `/var/log/` в текущую директорию.

2.4. Получение метрик по развёрнутому проекту

Для получения метрик Kubernetes можно использовать утилиту командной строки `kubectl`.

1. Получить информацию о доступных метриках для кластера: `kubectl top node`
2. Получить информацию о доступных метриках для сервиса: `kubectl top service`

3. Получить информацию о доступных метриках для проекта: `kubectl top deployment`

2.5. Изменение физических ресурсов, выделяемых на проекта

Для изменения физических ресурсов в Kubernetes можно использовать команды `kubectl`.

1. Изменение количества реплик:
`kubectl scale deployment <deployment-name> --replicas=<new-replica-count>`
2. Изменение ресурсных ограничений и запросов для пода:
`kubectl edit pod <pod-name>`

В открывшемся редакторе, необходимо найти раздел `resources` и изменить значения ресурсов (например, `cpu` и `memory`).